CivicFix — Your Complete Interview Prep Guide

A plain-English walkthrough of everything we built, why we built it that way, and how to talk about it out loud in an interview. Updated now that the project is essentially feature-complete — only deployment is left.

---

1. The Big Picture — What Is This App, Really?

Picture CivicFix as **three separate "workers" that talk to each other over the internet**:

1. **The Frontend (React)** — what a person sees in their browser: buttons, forms, pages. It cannot store anything permanently by itself.
2. **The Backend (Node.js + Express)** — the "brain." It receives requests like "create this issue" or "log this person in," decides if the request is allowed, and talks to the database.
3. **The Database (PostgreSQL)** — the permanent filing cabinet. It's the only place data survives after you close the browser or restart the server.

Core rule: these three pieces never talk to each other directly except through defined channels. The browser never touches the database directly — it always goes through the backend first.

The restaurant analogy (say this out loud in an interview — it lands well): the frontend is the dining room (what the customer sees and interacts with), the backend is the kitchen (where the real work and decisions happen, out of the customer's sight), and the database is the pantry (where ingredients — data — are permanently stored). The customer never walks into the kitchen and grabs food from the fridge themselves; they place an order (a request), and the kitchen decides what to do with it, including whether to say no.

---

2. The Tech Stack — And *Why* Each Piece Was Chosen

This table is one of the most interview-relevant things in this whole document. Be ready to explain **each row's "why,"** not just the row itself.

Layer	Choice	Why (in plain words)
Frontend	**React**	Builds UI out of reusable "components" (like Lego blocks) that automatically re-render when data changes. Extremely widely used, so it's a transferable, resume-relevant skill.
Backend	**Node.js + Express** (not NestJS)	**Deliberately chosen over NestJS.** NestJS adds structure (decorators, dependency injection, modules) that's great for large teams but hides *how* a request actually flows — bad for a first project where the goal is understanding fundamentals. Express keeps "request comes in → code runs → response goes out" fully visible in code you wrote yourself.
Database	**PostgreSQL**	A **relational database** — good because Users, Issues, and Upvotes are all genuinely *related* (a user *owns* issues, a user *upvotes* issues). Relational databases use foreign keys to connect tables (Section 4) and can enforce real-world rules — like "no duplicate upvotes" — at the database level itself, not just in application code.
Image storage	**Cloudinary** (free tier)	Databases handle large binary files (like photos) badly. Cloudinary stores the actual image and hands back a short text URL — Postgres only ever stores that URL, never the image itself.
Address lookup	**OpenStreetMap's Nominatim** (tried, then removed)	See Section 9's "map feature" story below — a great talking point about engineering judgment, not just a line in a table.
Login system	**JWT (JSON Web Token)**	Instead of asking the user to log in on every click, the server gives the browser a signed "ID card" it can show on future requests. Full explanation in Section 5.
Code history	**GitHub**	Every change is saved with a timestamp and

		message — nothing is ever truly lost, and it's the literal link you'll put on your resume/portfolio.
Frontend hosting	**Vercel** (planned, free tier)	Gives the React app a real, permanent public web address instead of only working on your own laptop.
Backend + DB hosting	**Render** (planned)	Keeps the Express server and Postgres database running 24/7.

"Why Express over NestJS" and "why PostgreSQL over something like MongoDB" are two of the most common interview questions for a project like this — you now have real, defensible answers above, not just "because I was told to."

---

3. The Full Journey of One Request — Two Concrete Examples

Reading a table of technologies doesn't teach you how the pieces fit together — watching a request travel through the whole system does. These are the two best stories to tell when asked "walk me through your architecture."

Example A: Logging in (the simpler story — start here if put on the spot)

1. **User types email + password into** `Login.jsx` and clicks "Log In." This is a **controlled input** — React tracks the exact current text of every box in a JavaScript object called `form`, updated on every keystroke.
2. **The form's** `onSubmit` **handler fires**, calling `fetch('http://localhost:3000/api/auth/login', { method: 'POST', body: JSON.stringify(form) })` — the frontend "placing an order," sending the email/password as **JSON** (a text format for structured data).
3. **Express receives it.** `app.use('/api/auth', authRoutes)` in `server/index.js` means: "any URL starting with `/api/auth` gets handed off to `routes/auth.js`." This is **routing** — matching a URL + method to the right code.
4. **Inside** `routes/auth.js`, the login route looks up the user by email using a **parameterized query** (`SELECT * FROM users WHERE email = $1`) — never gluing the typed email directly into a SQL string, which is the core defense against **SQL injection**. It then compares the submitted password against the stored hash with `bcrypt.compare()`.
5. If both checks pass, it creates a JWT (`jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '7d' })`) and sends it back as JSON.
6. **Back in the frontend**, the token gets saved into `localStorage` and into `AuthContext` — shared login state available to the whole app at once (more in Section 6) — which is what makes the Navbar immediately switch to showing "Hi, [name] / Log Out" instead of "Log In / Sign Up," with no page reload.
7. **On every future request that needs to know "who is this,"** the browser attaches that token as `Authorization: Bearer <token>`. The backend's `requireAuth` middleware checks it before letting the request through.

Example B: Reporting an issue with a photo (the richer story — use this if asked about the toughest part you built)

1. A logged-in visitor fills out the Report Issue form and picks a photo file, then submits.
2. Because a real file is involved (not just text), the frontend sends this as `multipart/form-data` instead of plain JSON — a request format built specifically for bundling files together with text fields, since JSON can't carry raw file bytes.
3. On the backend, the request passes through a chain of **middleware** — small functions that each get a chance to inspect or reject a request before the next one runs. `requireAuth` checks the JWT first; if it's missing or invalid, everything stops right here. Then `uploadMiddleware` (built on a library called **multer**) reads the incoming photo into memory as raw bytes, and checks it's really an image (by MIME type *and*, as a backup, by file extension — a real bug I hit and fixed, see Section 9).
4. If a photo was included, the backend hands those bytes to **Cloudinary**, which stores it and returns a URL.
5. The backend runs a Postgres `INSERT`, saving the title, category, location, description, that Cloudinary URL, and the reporting user's **real ID taken from the verified JWT — never from anything the browser claimed in the request body.** (If the server trusted a `user_id` field sent by the client instead, anyone could impersonate any other user just by editing a number in the request.)
6. Postgres confirms the save and returns the new row (with its auto-generated `id`); the backend forwards that to the frontend, which redirects to the new issue's detail page.

Every other feature (signup, browsing the feed, upvoting, filtering, changing status) follows this same shape: **frontend collects input → sends an API request → backend validates and enforces rules → backend talks to Postgres (and sometimes Cloudinary) → backend responds → frontend updates the screen.**

---

4. Database Design — The Relational Model, Explained

What "relational" actually means

Picture three separate spreadsheets instead of one giant one:

- `users` — one row per person: `id, name, email, password_hash, role, created_at`

- `issues` — one row per reported problem, including a `user_id` column saying *which user reported it*
- `upvotes` — one row per "this user upvoted this issue" event, with both a `user_id` and an `issue_id`

The connecting column (`user_id` inside `issues`) is called a *foreign key*. It doesn't copy the user's name or email into every issue row — it stores a *reference* (the user's `id`) pointing back to `users`. This is the entire idea behind a relational database: one authoritative copy of data, linked to wherever it's needed, instead of copied everywhere.

`UNIQUE(issue_id, user_id)` on `upvotes` makes it *physically impossible* at the database level for the same user to upvote the same issue twice, even if there were a bug in the backend code. Postgres itself throws error code `23505` ("unique_violation") when this is violated, which `routes/issues.js` specifically catches and turns into a friendly `409` response instead of a crash.

JOINs — pulling data from more than one table at once

Two real JOINs exist in this project, and both are good interview material:

- **Upvote counts**: every issue-listing query does a `LEFT JOIN` against `upvotes` and `COUNT()`s the matches, grouped per issue — `LEFT JOIN` (not a plain `JOIN`) specifically because an issue with **zero** upvotes still needs to show up with a count of 0, rather than disappearing from the results entirely.
- **Reporter name**: the single-issue query (`GET /api/issues/:id`) does a plain `JOIN` against `users` to pull in the reporter's name as `reporter_name`. This one's a plain `JOIN`, not `LEFT JOIN`, because every issue is *guaranteed* to have a real user attached — reporting requires being logged in — so there's never a case with no match to worry about losing.

Why `password_hash` and not `password`

We never store what someone actually typed — only the output of running it through **bcrypt**, a one-way scrambling algorithm. There's no function to reverse a hash back into the original password; checking a login means hashing the *newly typed* password the same way and comparing the two scrambled results. **Even if the entire database were stolen, the attacker would not have anyone's actual password.**

The database schema is real, runnable code — not just memory

Early on, these tables were created by hand-typing SQL into `psql`. That's now been replaced: `server/db/schema.sql` holds the real `CREATE TABLE` statements, and `npm run db:setup` runs them. It's written to be **idempotent** — safe to run more than once without error, using `CREATE TABLE IF NOT EXISTS` — so the exact same file can build a brand-new empty database from scratch (like Render's future production database) just as safely as it does nothing on a database that already has these tables. This is a genuinely good thing to mention if asked "how would your database get set up in production?"

---

5. Authentication Deep Dive — JWT, Middleware, and Roles

The problem JWT solves

Without a login system, the server has no memory between requests — every request would need to re-prove identity from scratch. That's both insecure and annoying.

How JWT solves it

1. On successful login, the server creates a token — a long string **signed** using a secret key only the server knows (`process.env.JWT_SECRET`, stored in `.env`, never committed to GitHub).
2. That token contains a small amount of information (`{ id, role }`) plus a 7-day expiry.
3. The browser stores this token and attaches it to future requests.
4. **The server never has to "remember" who's logged in** — it just re-verifies the signature each time. Anyone can *decode* a JWT and read what's inside (it's not secret content), but nobody can *forge* a new valid one without the server's secret key, which is what makes it trustworthy.

Middleware — the concept

Middleware is a function that runs between a request arriving and your actual route logic, with the power to either continue (`next()`) or stop the request right there. Think of it as a security checkpoint before you're allowed into a room.

- `requireAuth` — checks for a valid token. If missing or invalid, rejects with `401` before your route code ever runs. If valid, attaches the decoded info to `req.user`.
- `requireAdmin` — checks `req.user.role === 'admin'`. Must always run *after* `requireAuth`, since it depends on `req.user` already being set:

```
router.patch('/:id/status', requireAuth, requireAdmin, async (req, res) => { ... })
```

Two middlewares chained left to right — `requireAuth` proves *who* you are, `requireAdmin` then checks *what you're allowed to do*. If either fails, the route function never runs at all.

---

6. The Frontend — React Concepts in Practice

Components, JSX, and routing

A **component** is a reusable chunk of UI defined as a JavaScript function returning **JSX** (JavaScript XML — markup written directly inside JS). `Navbar.jsx` is written once and reused on every page.

`react-router-dom` maps a URL path (like `/report`) to a component without reloading the whole page from the server — only the part of the screen that changed actually updates. This is what makes CivicFix a **Single Page Application (SPA)**.

Shared login state: `AuthContext`

Early in this project, the Navbar had no idea whether someone was logged in — a known gap at the time. That's now fixed with **React Context**: `AuthContext.jsx` stores `token`, `user`, and `isLoggedIn` at the top of the whole app, readable by any component with one line (`useAuth()`) instead of manually passing that data down through every layer in between ("prop drilling"). It also checks `localStorage` on load, so a visitor stays logged in across page refreshes.

This same shared state is what decides which page shows at the root URL `/`: logged-out visitors see the Landing Hero marketing page; logged-in visitors see the real issue feed (`App.jsx`: `isLoggedIn ? <Home /> : <LandingHero />`).

State and controlled inputs

`useState` is a React "hook" — a function letting a component remember a value between renders and automatically re-render when it changes. Every form field, loading spinner, or fetched list of issues is state.

Talking to the backend: `fetch`

The pattern used throughout: send the request → `await` the response → check `response.ok` (true only for 2xx status codes) → if not ok, read `data.error` and show it → if ok, update the screen. The `try/catch/finally` structure matters: `catch` only fires if the network request fails entirely (server not running), giving a different message than a request that *reached* the server but was rejected. `finally` always runs regardless, used to turn off a "submitting..." state either way.

The Home feed's filter bar

`Home.jsx` holds `category` and `locationSearch` as state, and sends them to the backend as URL query parameters (`?category=...&location=...`) — filtering happens **server-side**, not by fetching everything and hiding rows in the browser, matching the "trust the server" pattern used everywhere else in this app. The location search box is **debounced**: instead of firing a network request on every single keystroke, a short timer resets on each keystroke and only actually fires once typing pauses — a common real-world performance pattern worth naming by term if asked how you'd avoid hammering a server with requests.

The Landing Hero page — the most technically interesting frontend piece

This is worth walking through if you want to show depth beyond CRUD forms. Logged-out visitors land on one fixed screen — the browser's normal scrollbar is disabled, and scrolling/swiping is instead captured and converted into a 0-to-1 "progress" number that crossfades the hero text into a "How It Works" panel over a static blurred background image.

The one genuinely tricky technical detail: this is built with the browser's **native** `addEventListener('wheel', ..., { passive: false })`, not React's `onWheel` JSX prop. React (and browsers by default) treat scroll listeners as "passive" for performance reasons, meaning `event.preventDefault()` on them can be silently ignored — which breaks exactly the "take over scrolling" effect this page needs. Explicitly passing `{ passive: false }` on a manually-attached listener is what actually makes `preventDefault()` work. A visitor whose OS has "reduce motion" turned on gets a plain, normally-scrolling fallback instead, via the `prefers-reduced-motion` media query — an accessibility detail worth mentioning unprompted.

---

7. Security Ideas Worth Remembering (High-Value Interview Material)

These four ideas show up constantly in interviews for *any* backend role, not just this project:

1. **Never trust the client.** Any data that determines *permission* (like "who am I") must come from something the server verified itself (the JWT), never from a value the browser simply claims.
2. **Never store plaintext passwords.** Always hash with something purpose-built like bcrypt, never plain storage or reversible encryption.
3. **Parameterized queries prevent SQL injection.** Never build a SQL string by directly gluing in user input — this includes the filter bar's `ILIKE` search, not just login.
4. **Vague error messages for authentication failures.** "Invalid email or password" (rather than two different messages for "no such email" vs. "wrong password") prevents an attacker from discovering which emails are registered just by trying to log in.

---

8. What's Actually Built Right Now (Honest Status)

Backend: feature-complete. Signup/login, full issue CRUD (create with photo upload, list with server-side filtering, list-mine, get-one with reporter name, upvote, admin status update) — every route manually tested through Postman before moving on.

Frontend: feature-complete. Navbar (with real login-aware state), Signup, Login, Landing Hero, Home feed (with filter bar), Issue Detail, Report Issue, My Reports — all built and styled to a single design system (`design.md`).

Database: three related tables, schema now real runnable code (`schema.sql` + `npm run db:setup`), not just hand-typed commands.

Not started: deployment. Frontend → Vercel, backend + database → Render. This is the one remaining piece of the whole project.

---

9. Likely Interview Questions — And How to Answer Them From This Project

LIKELY INTERVIEW QUESTION

"Walk me through your architecture." → Restaurant analogy (Section 1), then narrate Example A or B from Section 3 as a concrete story, not just a list of technologies.

LIKELY INTERVIEW QUESTION

"Why Express instead of a framework like NestJS?" → Beginner project; wanted every request's flow visible and self-written rather than hidden behind decorators/dependency injection — a deliberate learning tradeoff.

LIKELY INTERVIEW QUESTION

"Why PostgreSQL and not MongoDB?" → The data is inherently relational — foreign keys and a `UNIQUE` constraint let the database itself enforce rules like "no duplicate upvotes," rather than relying on application code to catch it.

LIKELY INTERVIEW QUESTION

"How does your login system work?" → Bcrypt-hashed passwords, JWT issued on successful login, verified via middleware on protected routes, `req.user` populated only from a cryptographically verified token — never trusted from client input.

LIKELY INTERVIEW QUESTION

"How did you prevent SQL injection?" → Parameterized queries (`$1`, `$2` placeholders) via the `pg` library everywhere user input touches a query — including the filter bar's search, not just auth.

LIKELY INTERVIEW QUESTION

"Tell me about a time you had to make a tradeoff, or a feature that didn't work out." (This is a genuinely great question to have a real answer for — most beginner-project answers here are made up. Yours isn't.) → *"I built a feature adding an interactive map to the issue detail page — geocoding the typed address into real coordinates with a free API, and rendering it with Leaflet. I got it fully working end to end, but ran into enough friction with it in practice that I made the call to back it out entirely rather than ship something half-working. I removed the code cleanly, updated my own project documentation to reflect the decision, and committed that as its own clear step in my Git history rather than pretending it never happened."* This shows you can build something *and* recognize when to cut it — a real engineering skill, not just a coding one.

LIKELY INTERVIEW QUESTION

"How did you approach the frontend visually — did you just start coding pages?" → No — every visual page went through a mockup step first, reviewed and approved before any real code was written, following a single documented design system (colors, type, spacing) kept in its own file so the whole app stays visually consistent rather than each page looking like it was designed separately.

LIKELY INTERVIEW QUESTION

"What would you do differently, or what's still missing?" → Deployment is the one honest gap — the app currently only runs on your own machine. Also worth mentioning: location is still plain text with no real coordinates (the map attempt is why), and there's no automated test suite yet, everything's been manually verified through Postman. Naming these clearly is a *good* sign to interviewers — it shows self-awareness about production-readiness, not just "getting it working."

---

10. Quick Glossary (for anything above that felt fuzzy)

- **API (Application Programming Interface):** the set of URLs/rules your backend exposes for other programs (like your React app) to request data or trigger actions.
- **REST:** a common API style built around standard HTTP methods (`GET` read, `POST` create, `PATCH` update) and predictable URLs representing "things," like `/api/issues/5`.
- **Endpoint:** one specific URL + method combination on your API, e.g. `POST /api/auth/login`.
- **Middleware:** code that runs in between a request arriving and your main route logic, able to block or allow it.
- **JOIN:** a single database query that pulls in matching rows from more than one table at once, instead of the backend making separate queries and stitching results together itself.
- **Foreign key:** a database-enforced rule that one table's column must reference a real row in another table.
- **Idempotent:** safe to run more than once without changing the outcome after the first time — used to describe `schema.sql`.
- **Schema:** the defined structure of a database table — which columns exist, what type each one is.

- **Environment variable:** a secret or config value (like a database password or JWT secret) kept outside your code, in a `.env` file, never committed to GitHub.
- **Token:** a piece of data (here, a JWT) that proves something ("this is user #5, logged in") without re-checking a password every request.
- **Debounce:** delaying an action (like a search request) until a short pause in activity, so it doesn't fire on every single keystroke.
- **Repository ("repo"):** a project's folder as tracked by Git/GitHub, with full history of every change.
- **Deployment:** making your app run on a public server that's always on, instead of only your own laptop.
- **CORS (Cross-Origin Resource Sharing):** a browser security rule blocking a webpage from one address from freely talking to a server at a different address unless the server explicitly allows it.
- **Hook (React):** a special function (like `useState`) letting a function-based component "hook into" React features like memory.
- **Signed upload (Cloudinary):** the backend validates and forwards a file to Cloudinary itself, rather than letting the browser upload directly — keeps Cloudinary's secret key off the frontend entirely.

---

11. If You're Asked to Draw the Architecture on a Whiteboard

Draw three boxes left to right, with arrows only in these directions:

```
[ Browser: React frontend ]  --HTTP request (JSON or multipart)-->  [ Node.js + Express backend ]
[ Browser: React frontend ]  <--HTTP response (JSON)---------------  [ Node.js + Express backend ]

[ Express backend ]  --parameterized SQL query-->  [ PostgreSQL database ]
[ Express backend ]  <--rows back------------------  [ PostgreSQL database ]

[ Express backend ]  --upload photo bytes-->  [ Cloudinary ]
[ Express backend ]  <--photo URL back------  [ Cloudinary ]
```

Say out loud while drawing it: *"The browser never talks to Postgres or Cloudinary directly — every arrow into either of those two only ever starts from my Express server. That's deliberate: it means every rule about who's allowed to do what lives in exactly one place I control, not scattered across the frontend where anyone could tamper with it."*